

Stack ADT, Operations & Array Implementation

Q. Define Stack as an Abstract Data Type (ADT). Explain its primary operations and demonstrate how a Stack can be implemented using an Array with suitable code snippets.

★ Definition to Remember

A **Stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle — the last element inserted is the first to be removed. All insertions and deletions occur at a single end called the **Top**. In an array implementation, **top = -1** represents an empty stack.

1. Stack as an ADT

Operation	Description	Condition
push(element)	Add element to the Top	Stack Overflow if full
pop()	Remove and return Top element	Stack Underflow if empty
peek() / top()	View top element without removing	Underflow if empty
isEmpty()	Returns True if stack has no elements	top == -1
isFull()	Returns True if stack is at max capacity	top == MAX-1

2. Stack Diagram

Initial (Empty):

top = -1

After push(10, 20, 30):

top = 2

30 ← top

20

10

3. Array Implementation in C

```
#include <stdio.h>
#define MAX 5

int stack[MAX];
int top = -1;

void push(int value) {
    if (top == MAX - 1)
        printf("Stack Overflow!\n");
    else
        stack[++top] = value;    // increment top, then insert
}

int pop() {
    if (top == -1) {
        printf("Stack Underflow!\n");
        return -1;
    }
    return stack[top--];        // return top element, then decrement
}

int peek() {
    if (top == -1) return -1;
    return stack[top];
}

int isEmpty() { return top == -1; }
```

4. Advantages and Limitations

Array Implementation	
Advantage	Simple, fast ($O(1)$ for push/pop/peek), contiguous memory
Disadvantage	Fixed size — cannot grow dynamically (use Linked List for dynamic stack)

★ Must-Write Points (for 10 marks)

- Stack follows **LIFO** (Last In, First Out) — all operations at the Top.
- Five operations: **push, pop, peek, isEmpty, isFull**.
- Stack Overflow**: push on a full stack; **Stack Underflow**: pop from an empty stack.
- Array implementation: **top** variable tracks the top index; **top = -1** means empty.
- Push: **stack[++top] = value** ; Pop: **return stack[top--]** .
- All stack operations (push/pop/peek) are **$O(1)$** time complexity.
- Limitation of array implementation: **fixed size** — cannot grow dynamically at runtime.

⚡ Quick Recall

Stack → LIFO → Top pointer → push (Overflow) → pop (Underflow) → peek → isEmpty (top==-1) → Array: $O(1)$, fixed size



Complete Executable C Code: Array Implementation of Stack ADT

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

struct Stack {
    int items[MAX];
    int top;
};

void initStack(struct Stack *s) {
    s->top = -1;
}

int isFull(struct Stack *s) {
    return s->top == MAX - 1;
}

int isEmpty(struct Stack *s) {
    return s->top == -1;
}

void push(struct Stack *s, int val) {
    if (isFull(s)) {
        printf("Stack Overflow! Cannot push %d\n", val);
        return;
    }
    s->items[++(s->top)] = val;
    printf("Pushed %d onto stack\n", val);
}

int pop(struct Stack *s) {
    if (isEmpty(s)) {
        printf("Stack Underflow! Cannot pop\n");
        return -1;
    }
    return s->items[(s->top)--];
}

int peek(struct Stack *s) {
    if (isEmpty(s)) return -1;
    return s->items[s->top];
}

void display(struct Stack *s) {
    if (isEmpty(s)) {
        printf("Stack is Empty\n");
        return;
    }
    printf("Stack (Top to Bottom): ");
    for (int i = s->top; i >= 0; i--) {
        printf("%d ", s->items[i]);
    }
    printf("\n");
}
```

```
}  
  
int main() {  
    struct Stack s;  
    initStack(&s);  
  
    push(&s, 10);  
    push(&s, 20);  
    push(&s, 30);  
    display(&s);  
  
    printf("Popped Value: %d\n", pop(&s));  
    display(&s);  
  
    printf("Top Element (Peek): %d\n", peek(&s));  
    return 0;  
}
```